

Problem J1. Cupcake Party

Read the number of regular boxes and small boxes, then compute the total number of cupcakes directly. A regular box contributes 8 cupcakes and a small box contributes 3 cupcakes, so the number left after the class receives 28 cupcakes is

$$8R + 3S - 28.$$

No loops or data structures are needed.

Complexity. The running time is $O(1)$ and the memory usage is $O(1)$.

Problem J2. Fergusonball Ratings

Process the players one at a time. For each player, read the number of points and fouls, compute the star rating

$$5P - 3F,$$

and count the player if this value is greater than 40. After all players have been processed, print the count. If every player was counted, append the plus sign required by the statement.

The main implementation detail is to read exactly two lines of data for each of the N players and to avoid printing extra spaces or lines.

Complexity. The running time is $O(N)$, and only a few integer variables are needed.

Problem J3. Harp Tuning

The input is a compact sequence of instructions. Each instruction has three parts: a string of letters naming the string, either + or -, and a positive integer telling how many turns to make. The task is therefore a tokenization problem.

Scan the input from left to right while maintaining the current instruction name. When a sign is found, it determines whether the output word is **tighten** or **loosen**. Then read all following digits as one integer, output the translated instruction, and continue scanning the remaining characters.

A state-machine implementation is also natural: the current state is one of “reading letters”, “reading the sign”, or “reading digits”. Outputting each translated instruction as soon as it is recognized avoids storing all instructions.

Complexity. Each character is examined a constant number of times, so the running time is linear in the input length.

Problem J4. Group Work

Store all constraints first, because the actual group assignment is given later. After the groups are read, build a map from each student’s name to the index of the group containing that student.

For a must-be-together constraint (a, b) , the constraint is violated exactly when a and b have different group indices. For a must-not-be-together constraint, it is violated exactly when the two students have the same group index. Count all violated constraints.

A common mistake is to assume that names appear in a convenient order or that a group can violate at most one rule. The group lookup should be direct, for example by using a dictionary, hash map, or unordered map.

Complexity. With hash-map lookups, the running time is $O(X + Y + G)$, where X and Y are the numbers of constraints and G is the total number of listed students.

Problem J5. Square Pool

For very small cases, brute force over all possible pool positions and sizes is sufficient. For the full task, the important observation is that an optimal square pool can be shifted upward until its top side hits either the yard boundary or a tree, and shifted left until its left side hits either the yard boundary or a tree. Thus the top coordinate and the left coordinate of an optimal square come from only $T + 1$ candidates each, where T is the number of trees.

Enumerate all candidate top positions and all candidate left positions. For each candidate corner, determine the largest side length that still stays inside the yard and does not contain a tree. This can be done by checking all trees and taking the minimum blocking distance from a tree that lies down-right from the chosen corner.

This changes the search from depending on the yard side length to depending mainly on the number of trees, which is the intended way to handle very large yards.

Complexity. Enumerating $O(T^2)$ candidate corners and checking $O(T)$ trees for each gives $O(T^3)$ time and $O(T)$ memory.